
Gain standard: PLC software structure

GAIN

Automation
Technology

Document name: Gain standard v2.0 - PLC software structure
Status: Final

Released by: Gain Automation Technology B.V.
hereafter named Gain

Author: Dieter Kuiper
Last modified by: Dieter Kuiper
Date: 01-05-2022

Table of content

1.	INTRODUCTION	3
1.1.	REVISION HISTORY	3
1.2.	HISTORY AND CONTEXT	3
2.	GAIN PLC SOFTWARE STANDARD	4
3.	PLC SOFTWARE STRUCTURE	5
3.1.	HIERARCHICAL LAYERS	5
3.1.1.	<i>Management functions</i>	5
3.1.2.	<i>Units (UN)</i>	5
3.1.3.	<i>Equipment modules (EM)</i>	6
3.1.4.	<i>Control modules (CM)</i>	7
3.2.	GLOBAL AREA	8
3.2.1.	<i>Auxiliaries</i>	8
3.2.2.	<i>Global data</i>	8
APPENDIX A:	HIERARCHICAL PROGRAM OVERVIEW	9
APPENDIX B:	CASE STUDY	10
B.1	MAIN PROGRAM	10
B.2	MANAGEMENT LAYER	10
B.2.1	<i>Alarm management</i>	11
B.2.2	<i>Mode management</i>	11
B.2.3	<i>Recipe management</i>	11
B.3	UNIT LAYER	11
B.3.1	<i>UN_Machine</i>	11
B.4	EQUIPMENT MODULES	14
B.4.1	<i>EM_Safety</i>	15
B.4.2	<i>EM_Uilities</i>	15
B.4.3	<i>EM_Conveyor</i>	15
B.5	CONTROL MODULES.....	20
B.5.1	<i>CM_Motor</i>	20

1. Introduction

1.1. Revision history

Version	Date	Status	Comment	Initials
0.1	02-04-2021	Concept		MCo
0.2	18-05-2021	Concept	Changed layer structure to S88 naming	CdK
1.0	05-08-2021	Approved		MDu
1.1	05-11-2021	Revision	Restructured document outline	DKu
2.0	01-05-2022	Final	Revised naming convention, implemented S88 naming	DKu
2.1	23-05-2022	Final	Watermark added	WBo

Table 1: Revision history

1.2. History and context

These are the coding guidelines for PLC projects. This document can be used by Gain engineers.

It is a 'living' document. Remarks on this document will be appreciated to extend the knowledge which is placed in this document.

gain

Automation
Technology

2. Gain PLC software standard

This document is part of the “Gain PLC software standard”. This standard is separated in several documents, each document describes a part of the standard. These documents may be used individually and combined with other documents to create a customer specific standard.

A complete PLC software standard should (at least) contain the following topics:

- PLC naming convention
- PLC software structure
- PLC coding convention
- PLC version control

In this document the “PLC software structure” is described in chapter 3. In Appendix A; a schematical overview of the standard hierarchy is provided. In Appendix B; an example PLC program is provided as a case study.

3. PLC software structure

The software is built up in a structured manner. Its overall architecture contains 4 hierarchical layers and a common area. This hierarchical design is based on the PackML and ISA-88 standard.

3.1. Hierarchical layers

The PLC software consists of four hierarchical layers:

1. Management layer
2. Unit layer
3. Equipment module layer
4. Control module layer

A graphical example of the hierarchical structure is shown in the next figure.

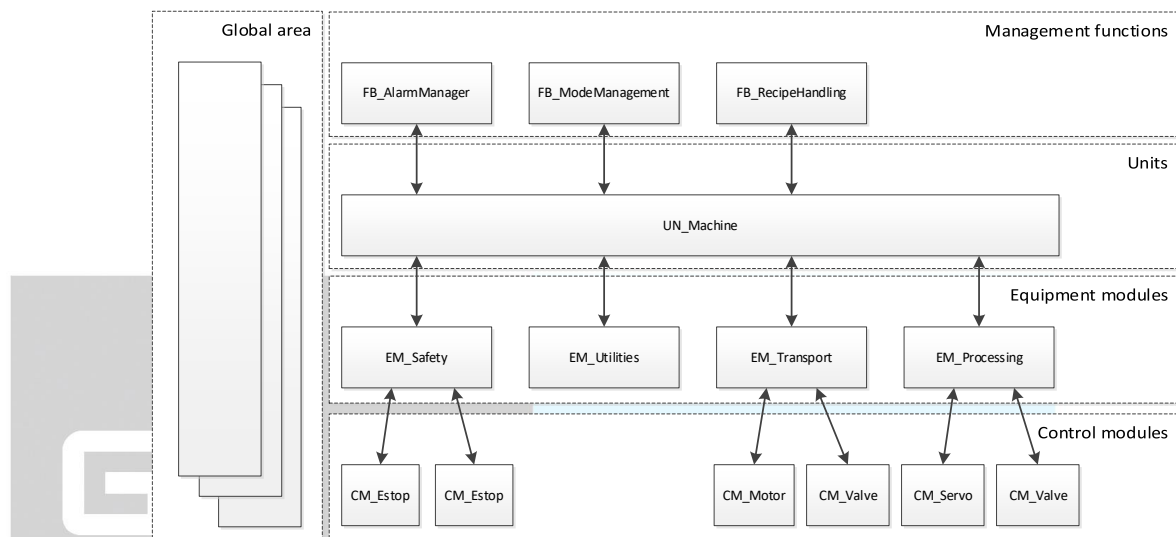


Figure 1 Hierarchical structure

3.1.1. Management functions

The management layer is a placeholder for functionalities which control and affect the data or overall state of the machine or installation. This layer manages tasks such as mode control (Manual, Auto, Standby), alarms, recipe handling, etc.

3.1.1.1. Rules for management layer:

1. Function blocks are declared and called at the highest level (main program).
2. Management functions are allowed to directly access global data.

3.1.2. Units (UN)

A unit represents (a part of) a machine or installation which has its an internal state controller. The unit itself should not contain any functional logic. It consists out of equipment and control modules to achieve its overall functional purpose. Small PLC projects generally have a single unit whereas larger projects can contain multiple units. Normally each unit has its own mode manager.

3.1.2.1.**Rules for Units:**

1. Units are declared and called at the highest level (main program).
2. Units can only communicate with its own children and other units.

3.1.2.2.**Software structure**

The internal code of a unit should be divided into sections/regions for readability. For consistency, these regions must be executed in a specific order:

No.	Region	Description
1	Init	Unit initialization, set initial data.
2	Inputs	Process unit inputs.
3	State handling	Determine PackML states and modes for all sub-modules
4	Modules	Call sub-modules (equipment and/or control modules)
5	Alarms	Collect alarm conditions from sub-modules.
6	Outputs	Process unit outputs.
7	Statistics	Update statistical data.
8	HMI state	Write relevant status/data towards HMI.

3.1.3.**Equipment modules (EM)**

The equipment modules contain the functional logic of the machine or installation. An equipment module is an isolated entity, which is responsible for a specific task (e.g., lifting a pallet, checking a product, transportation, etc.).

Complex equipment modules can consist out of one or more equipment modules. The parent module acts as manager over the child modules to fulfill the overall task.

3.1.3.1.**Rules for equipment modules**

1. EMs can only be called by a single parent (unit or equipment module)
2. EMs may only communicate with its children and with its parent.
3. EMs are allowed to access global data (preferably via auxiliaries, see 3.2.1).

3.1.3.2.**Software structure**

The internal code of an equipment module should be divided into sections/regions for readability. For consistency, these regions must be executed in a specific order:

No.	Region	Description
1	Init	Module initialization, set initial data.
2	Inputs	Read hardware and module inputs. If applicable, apply filtering, scaling and/or conversion to local variables.
3	State handling	Call FB to handle mode and state changes.
4	HMI control	Process inputs from HMI (e.g., buttons). If applicable, apply limitations and conversion to local variables.
5	Logic	Handle the internal logic of the equipment module.

6	Modules	Call sub-modules (equipment and/or control modules).
7	Alarms	Monitor alarms (based on logic/process variables).
8	Outputs	Set hardware and module outputs.
9	Statistics	Update statistical data.
10	HMI state	Write relevant status/data towards HMI.

3.1.4. Control modules (CM)

A control module is a passive element without 'intelligence'. It must be controlled by a higher-level module to function. In general, it is an implementation of a low-level physical device, such as a motor or valve.

Control modules can be part of another control module. This parent control module is a 'wrapper' for one or more child modules. For example, a position controller for a servo or a pneumatic arm with 3 positions (2 valves).

3.1.4.1. Rules for control modules

1. CMs can only be called by a single parent (equipment or control module)
2. CMs may only communicate with its children and with its parent.
3. CMs may not access global data.
4. CMs must not contain machine or installation specific logic.
5. CMs must not act on its own.

3.1.4.2. Software structure

The internal code of a control module should be divided into sections/regions for readability. For consistency, these regions must be executed in a specific order:

No.	Region	Description
1	Init	Module initialization, set initial data.
2	Inputs	Read hardware and module inputs. If applicable, apply filtering, scaling and/or conversion to local variables.
3	Alarms	Monitor alarms (based on input conditions).
4	HMI control	Process inputs from HMI (e.g., buttons). If applicable, apply limitations and conversion to local variables.
5	Logic	Handle the internal logic of the control module.
6	Outputs	Set hardware and module outputs.
7	Statistics	Update statistical data.
8	HMI state	Write relevant status/data towards HMI.

3.2. Global area

The global area is not part of the four hierarchical layers. The area is divided into two sections, the common functionalities (auxiliaries) and the global data.

3.2.1. Auxiliaries

Auxiliaries are common functions and function blocks which can be used throughout the entire program. These software blocks are 'helper functions' and should not contain any machine/installation specific logic.

3.2.1.1. Rules for auxiliaries

1. Auxiliaries are stand-alone functions and can only access global data and/or other auxiliaries.
2. An auxiliary should have a single purpose/functionality.

3.2.2. Global data

The global area is to collect data if necessary. These data can be:

- Communication between an HMI.
- Alarm table.
- Module states.
- Shift register.

Appendix A: Hierarchical program overview

A hierarchical overview of an example PLC program is shown below, it depicts the functional layers and the relationships between the elements.

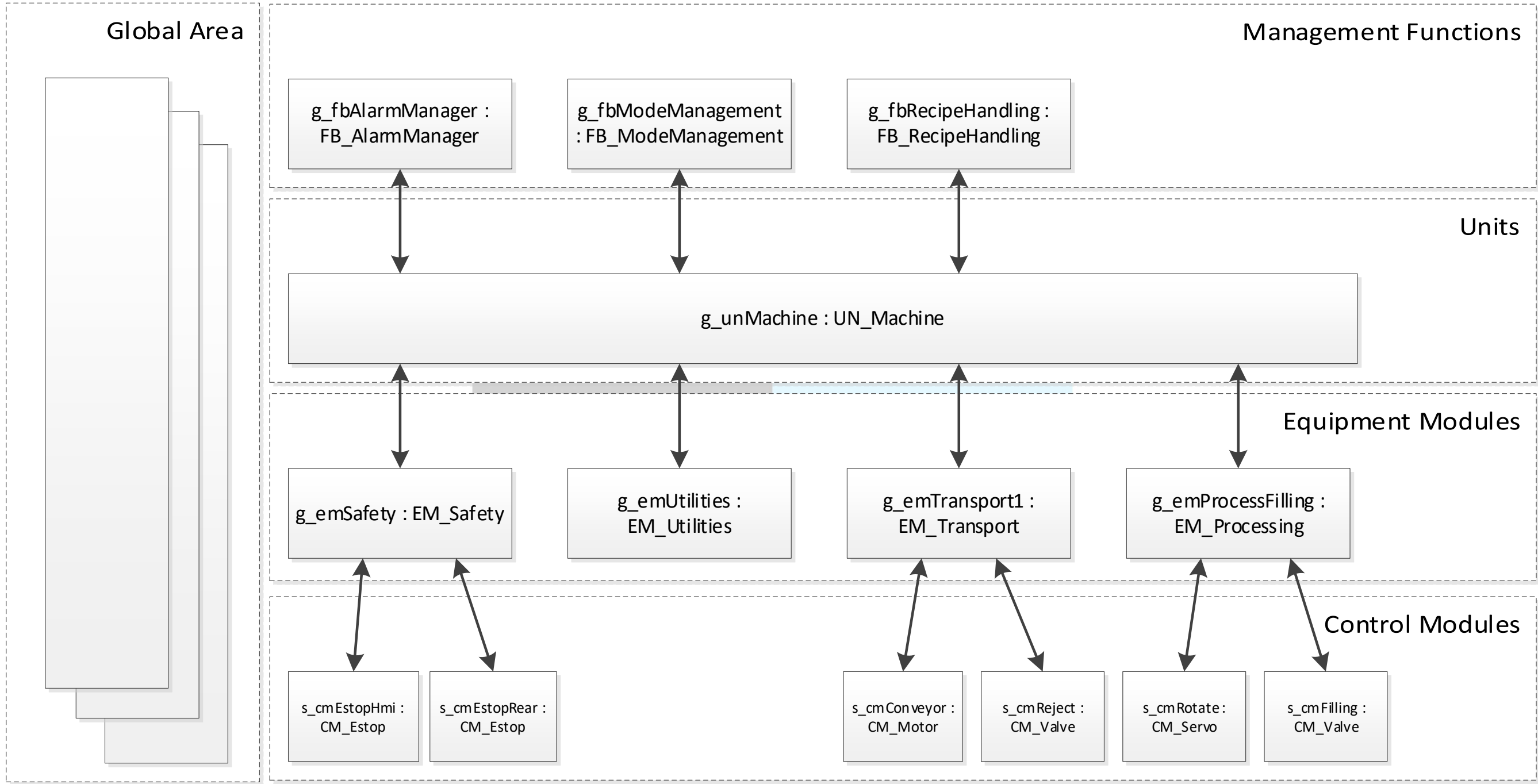


Figure 1: Hierarchical program overview

Appendix B: Case study

In this appendix an example program is used which implements the Gain PLC software structure and the Gain PLC naming convention. The general software structure can be seen in Appendix A:

It should be noted that all examples are in 'Pseudo code', the code is solely developed for illustrative purposes.

B.1 Main program

The 'main program' is the entry point of the PLC program. This program calls all the function blocks in the management layer and all units.

```
(*****
Management Layer
*****)
(* Process alarm management *)
g_fbAlarmManagement(
    i_stAlarmState := g_unMachine.q_stAlarmState,
    i_bResetAlarms := g_stIO.I_bResetAlarms,
    iq_stHmi:= g_stHmi.stAlarmManagement);

(* Process mode control *)
g_fbModeManagement(
    i_bReqStart := g_stIO.I_bStartButton,
    i_bReqStop := g_stIO.I_bStopButton OR g_fbAlarmManagement.q_bReqStop,
    i_bReqAbort := g_fbAlarmManagement.q_bReqAbort,
    i_bReqHeld := g_fbAlarmManagement.q_bReqHeld,
    i_bReqSuspend := g_g_fbAlarmManagement.q_bReqSuspend,
    i_bReqModeManual := g_stIO.I_bSwitchManualAuto,
    i_bReqModeProduction := NOT(g_stIO.I_bSwitchManualAuto),
    iq_stHmi := g_stHmi.stModeManagementI);

(*****
Unit Layer
*****)
(* Call unit machine *)
g_unMachine(
    i_eReqMode := g_fbModeManagement.q_eReqMode,
    i_eReqState := g_fbModeManagement.q_eReqState,
    i_bResetAlarms := g_fbAlarmManagement.q_bResetPulse,
    iq_stHmi:= g_stHmi.unMachine);
```

B.2 Management layer

The management layer (in this case study) consists out of 3 managers. Each has a specific function and operates independently.

```
(*****
Management Layer
*****)
```

```
g_fbAlarmManagement      : FB_AlarmManagement;
g_fbModeManagement       : FB_ModeManagement;
g_fbRecipeManagement     : FB_RecipeManagement;
```

B.2.1 Alarm management

The overall alarm handling consist out of two function blocks an 'alarm manager' and an 'alarm handler'. Generally, the PLC program contains a single manager and many alarm handlers. Every instance of an alarm handler represents a specific alarm condition.

During the start-up procedure, all alarm handlers retrieve their specific configuration from the alarm manager. When an alarm handler is activated (or de-activated), the alarm manager is notified. The alarm management maintains the list of active alarms. The list can be presented on an HMI.

B.2.2 Mode management

The mode manager manages the start, stop and mode selection of the machine. It can also provide status (Pack ML) information to external machines.

B.2.3 Recipe management

The recipe manager is generally used for product specific parameters. The manager observes the unit Pack ML states, when the units are (for example) stopped/idle, a new recipe can be selected for the next batch. The manager can be used to load/save recipes or provides an interface for an external recipe handler (for example an HMI).

B.3 Unit layer

The unit layer generally contains a single unit. Large installations/machines can have multiple units, see section 3.1.2. The units are responsible for calling their corresponding equipment modules.

```
(*****
Unit Layer
*****)

g_unMachine                : UN_Machine;
```

B.3.1 UN_Machine

In the following sections the inner working of this unit is studied. The unit consists of safety (B.4.1), utilities (B.4.2) and conveyor (B.4.3) equipment modules.

B.3.1.1 Inputs

In the inputs section of this unit the safety and utilities are called. Although they are equipment modules, they are considered as inputs for the unit. If the safety conditions are not OK (abort/stop alarms) the corresponding equipment modules must be aborted/stopped. The individual alarm states are also collected, they can be used to suppress cascading alarms.

```

(*****
Unit Inputs
*****
(* Process safety signals *)
g_emSafety(
    i_bResetAlarms := i_bResetAlarms,
    iq_stIO := g_stIO.emSafety,
    iq_stHmi := g_stHmi.emSafety );

(* Process non-safe input signals *)
g_emUtilities(
    i_bResetAlarms := i_bResetAlarms,
    iq_stIO := g_stIO.emUtilities,
    iq_stHmi := g_stHmi.emUtilities);

(* Collect Safety signals *)
s_stSafetyStatus.bPowerSupplyOK := g_emUtilities.q_bPowerSupplyOK;
s_stSafetyStatus.bEmStopsOK := g_emSafety.q_bEmStopsOK;
s_stSafetyStatus.bDoorsOK := g_emSafety.q_bDoorsOK;
s_stSafetyStatus.bAirPressureOK := g_emSafety.q_bAirPressureOK;

```

B.3.1.2

Alarms

In the alarms section, the alarm states of all child modules are collected and combined. In this scenario the Hold and Suspend alarms of the conveyors are ignored. This means that the unit does not change state (e.g., remains in execute) when a conveyor has a hold/suspend alarm. When any child module has a stop or abort alarm, the entire unit goes into the Stopping or Aborting state.

```

(*****
Alarms
*****
(* Collect Alarms *)
s_stAlarmState.bReqAbort := g_emSafety.q_stAlarmState.bReqAbort
                        OR g_emUtilities.q_stAlarmState.bReqAbort
                        OR g_emConveyor1.q_stAlarmState.bReqAbort
                        OR g_emConveyor2.q_stAlarmState.bReqAbort;
s_stAlarmState.bReqStop := g_emSafety.q_stAlarmState.bReqStop
                        OR g_emUtilities.q_stAlarmState.bReqStop
                        OR g_emConveyor1.q_stAlarmState.bReqStop
                        OR g_emConveyor2.q_stAlarmState.bReqStop;
s_stAlarmState.bReqHold := FALSE; // Ignore submodules held alarms
s_stAlarmState.bReqSuspend := FALSE; // Ignore submodules suspended alarms

```

B.3.1.3

HMI Control

This unit has no HMI control interface.

B.3.1.4**State handling**

In the state handling of the unit, the states and modes of all child modules are collected and fed into a general PackML function block. This function block determines the actual unit state and mode based on the requested state/mode, the alarm state, and the actual states/modes of the child modules.

```
(*****
  State control of unit
  *****)
(* Collect actual mode/states of all child modules *)
s_aeActChildModes[1] := g_emConveyor1.q_eActMode;
s_aeActChildModes[2] := g_emConveyor2.q_eActMode;
s_aeActChildStates[1] := g_emConveyor1.q_eActState;
s_aeActChildStates[2] := g_emConveyor2.q_eActState;

(* Call PackML handler for units *)
s_fbPackML(
    i_eReqMode := i_eReqMode,
    i_eReqState := i_eReqState,
    i_stAlarmState := s_stAlarmState,
    i_aeChildModes := s_aeActChildModes,
    i_aeChildState := s_aeActChildStates,
    q_eActMode := s_eActMode,
    q_eActState := s_eActState,);
```

B.3.1.5**Modules**

In the modules section, all child modules of the unit (equipment and/or control modules) are called. Also, the module interface signals (infeed/outfeed) are connected.

```

(*****
Unit sub-modules
*****)

(* Call equipment module for conveyor 1 *)
g_emConveyor1(
    i_bReqEnable := TRUE,
    i_bResetAlarms := i_bResetAlarms,
    i_bAlwOutfeed := g_emConveyor2.q_bAlwInfeed,
    i_eReqMode := s_fbPackML.q_eReqChildMode,
    i_eReqState := s_fbPackML.q_eReqChildState,
    i_stSafetyStatus := s_stSafetyStatus,
    iq_stIO := g_stIO.emConveyor1,
    iq_stHmi := g_stHmi.emConveyor1 );

(* Call equipment module for conveyor 2 *)
g_emConveyor2(
    i_bReqEnable := TRUE,
    i_bResetAlarms := i_bResetAlarms,
    i_bAlwOutfeed := TRUE,
    i_eReqMode := s_fbPackML.q_eReqChildMode,
    i_eReqState := s_fbPackML.q_eReqChildState,
    i_stSafetyStatus := s_stSafetyStatus,
    iq_stIO := g_stIO.emConveyor1,
    iq_stHmi := g_stHmi.emConveyor1 );

```

B.3.1.6

Outputs

This unit has no physical outputs, only the function block outputs are set.

```

(*****
Outputs
*****)

(* Function block outputs *)
q_bStaEnabled := i_bReqEnable;
q_eActState := s_eActState;
q_eActMode := s_eActMode;
q_stAlarmState := s_stAlarmState;

```

B.3.1.7

HMI status

This unit has no HMI status interface.

B.4

Equipment modules

The equipment modules are responsible for the functional logic of the machine/installation, see section 3.1.3. These modules can be declared local (in the unit) or global. It should be noted when declared global, the hierarchical rules should still be applied (Each module only communicates with its hierarchical parent/child).

```
(*****
Equipment Module Layer
*****)

(* General Equipment Modules *)
g_emSafety           : EM_Safety;
g_emUtilities        : EM_Utilities;

(* Equipment Modules Unit 1 *)
g_emConveyor1       : EM_TemplateModule;
g_emConveyor2       : EM_TemplateModule;
```

B.4.1 EM_Safety

The safety equipment module is used to collect and observe signals from a (external) safety controller. For example, when a safety door is opened this equipment module is responsible for triggering a stop/abort in the PLC program, before the safety controller deactivates the STO.

The status from the safety equipment module can be used to suppress cascading errors and can prevent the unit from starting when safety is not ok.

B.4.2 EM_Uilities

The utilities equipment module is similar to the safety equipment module. The difference is that this module is responsible to observe non-safe signals. These can include for example: power supplies, fuses, air pressure, temperature sensors, etc. When conditions are not ok, a controlled stop or an abort can be triggered.

The status from the utility's equipment module can be used to suppress cascading errors and can prevent the unit from starting when conditions are not ok.

B.4.3 EM_Conveyor

In the following sections the inner working of this equipment module is studied. This module consists of 1 digital input (production detection) and 1 control module.

B.4.3.1 Initialization of module

The initialization of the configuration settings is only done once (default values).

```

(*****
  Initialisation of module
  *****)
IF NOT(s_bInitDone) THEN
  (* Set initial values for configurations *)
  s_stCfgProductDetection.bEnabled := TRUE;
  s_stCfgProductDetection.tmDelayOn := T#10ms;

  s_stCfgAlmProductStuck.bEnabled := TRUE;
  s_stCfgAlmProductStuck.tmDelayOn := T#5000ms; // Generate alarm after 5sec

  s_stCfgMotor.stThermalProction.bEnabled := TRUE;
  s_stCfgMotor.stThermalProction.tmDelayOn := T#10ms;

  s_stCfgMotor.stRunningFeedback.bEnabled := TRUE;
  s_stCfgMotor.stRunningFeedback. tmDelayOn:= T#10ms;

  s_bInitDone := TRUE;
END_IF;

```

B.4.3.2**Inputs**

This equipment module has one physical hardware input, the sensor at the of the conveyor belt. The IO of the motor is handled by the control module, see B.5.1.

```

(*****
  Inputs
  *****)
(* Input filtering of product detection at the end of the conveyor belt *)
s_fbProductDetection(
  I_bInput := iq_stIO.I_bProductDection,
  i_stConfig := s_stCfgProductDetection,
  iq_stHmi := iq_stHmi.stProductDetection );

```

B.4.3.3**State handling**

In this section the actual state and mode of the equipment module is determined. Based upon the external request (from parent unit/equipment module) and the internal alarm state, the actual state and mode is set. In this case a standard PackML function block is used.


```

(*****
State control of module
*****)
s_fbPackML(
    i_eReqMode := i_stControl.eReqMode,
    i_eReqState := i_stControl.eReqState,
    i_bActStateComplete := (s_eStep = E_EM_Conveyor.Idle),
    i_stAlarmState := s_stAlarmState,
    q_eActMode := s_eActMode,
    q_eActState := s_eActState,

```

B.4.3.4 HMI control

This equipment module has no HMI control functionalities, manual control of the motor is already integrated into the control modules, see B.5.1.4.

B.4.3.5 Logic

In the logic section the functionalities of the equipment module are programmed. In this scenario we have a conveyor that transports product to the next equipment module. When the machine is running (in execute) the motor is enabled. When outfeed is allowed (by the next module) we can run the motor. When the outfeed is not allowed, we run the motor until we see a product at the sensor. Then we stop the motor and wait until the outfeed is allowed (or product has been removed).

```

(*****
Logic
*****)
CASE (s_eStep) OF
E_EM_Conveyor.Idle:
    s_cmMotor.i_bReqEnable := FALSE;
    s_cmMotor.i_bReqRun := FALSE;
    s_cmMotor.i_bReqStop := FALSE;

    IF i_bEnable AND (s_eActState = E_State_Execute) THEN
        s_eStep = E_EM_Conveyor.Running;
    END_IF

E_EM_Conveyor.Running:
    s_cmMotor.i_bReqEnable := TRUE;
    s_cmMotor.i_bReqRun := (s_eActMode = E_Mode_Production);
    s_cmMotor.i_bReqStop := FALSE;

    IF NOT(i_bEnable) OR NOT(s_eActState = E_State_Execute) THEN
        s_eStep = E_EM_Conveyor.Stopping;
    ELSIF NOT(i_bAlwOutfeed) AND s_fbProductDetection.q_bStaActive THEN
        s_eStep = E_EM_Conveyor.WaitForOutfeed;
    END_IF

E_EM_Conveyor.WaitForOutfeed:
    s_cmMotor.i_bReqEnable := TRUE;
    s_cmMotor.i_bReqRun := FALSE;
    s_cmMotor.i_bReqStop := TRUE;

    IF NOT(i_bEnable) OR NOT(s_eActState = E_State_Execute) THEN
        s_eStep = E_EM_Conveyor.Stopping;
    ELSIF i_bAlwOutfeed OR NOT(s_fbProductDetection.q_bStaActive) THEN
        s_eStep = E_EM_Conveyor.Running;
    END_IF

E_EM_Conveyor.Stopping:
    s_cmMotor.i_bReqEnable := TRUE;
    s_cmMotor.i_bReqRun := FALSE;
    s_cmMotor.i_bReqStop := TRUE;

    IF NOT(s_cmMotor.q_bStaRunning) THEN
        s_eStep = E_EM_Conveyor.Idle;
    END_IF
END_CASE

```

B.4.3.6

Modules

In the modules section all sub-modules (equipment or control modules) are called. In this equipment module there is only one control module, the motor.

```

(*****
Modules
*****
(* Control Module Motor *)
s_cmMotor(
    i_bReqEnable := i_stSafetyStatus.bPowerSupplyOK,
    i_stConfig := s_stMotorConfig,
    iq_stIO := iq_stIO.stMotor,
    iq_stHmi := iq_stHmi.stMotor);

```

B.4.3.7**Alarms**

In the alarms section we collect all alarms from the sub-modules and determine the alarm state. This condition may affect the current PackML state of the module.

```

(*****
Alarms
*****
(* Product stuck in the sensor at the end of the conveyor *)
s_fbAlmProductStuck(
    i_bTrigger := i_bEnable AND s_emMotor.q_bStaRunning
                AND s_fbProductDetection.q_bStaActive,
    i_bReset := i_bResetAlarms,
    i_stConfig := s_stCfgAlmProductStuck,
    q_stStaActive := q_stAlarms.bProductStuck);

(* Collect Alarms *)
s_stAlarmState.bReqAbort := FALSE;
s_stAlarmState.bReqStop := s_emMotor.q_bStaError; // We cannot continue
s_stAlarmState.bReqHold := q_stAlarms.bProductStuck; // Continue if product is removed

```

B.4.3.8**Outputs**

This equipment module does not have physical hardware outputs, the motor outputs are handled by the control module, see B.5.1.6. In this case only the function block outputs of the equipment module are set.

```

(*****
Outputs
*****
(* Function block outputs *)
q_bStaEnabled := i_bReqEnable;
q_eActState := s_eActState;
q_eActMode := s_eActMode;
q_stAlarmState := s_stAlarmState;
q_bAlwInfeed := (s_eStep = E_EM_Conveyor.Running);

```

B.4.3.9**Statistics**

In this example there is only one statistical value, the number of products detected.

```

(*****
Statistics
*****
(* Count products on raising edge signal only when motor is running *)
s_fbProductCounter(
    i_bCount := s_fbProductDetection.q_bStaActiveOsr AND s_cmMotor.q_bStaRunning,
    i_bReset := s_bResetStatistics );

```

B.4.3.10 HMI status

In this section all relevant data and status information is processed and copied to an HMI friendly structure. In this example we have 1 LED, a statistical value and status.

```

(*****
Hmi Status
*****
(* Hmi LED product detection *)
s_fbHmiLedProductDetection(
    i_bStatus := s_fbProductDetection.q_bStaActive,
    iq_stHmi := iq_stHmi.stLedProductDetection );

(* Copy status information to HMI *)
iq_stHmi.fStatProductCounter := s_fbProductCounter.q_nValue;
iq_stHmi.eStep := s_eStep;
iq_stHmi.eState := s_eActState;

```

B.5 Control modules

In this example there are two control modules (the motors of the conveyor), both are instances of CM_Motor.

B.5.1 CM_Motor

In the following sections the inner working of this control module is studied. This motor consists of 2 digital input and 1 digital output.

B.5.1.1 Initialization of module

The initialization is only done once during the first call of the module. During this process the initial values of local variables (if applicable) are determined.

```

(*****
Initialisation of module
*****
IF NOT(s_bInitDone) THEN
    (* Set initial values of local variables *)
    s_bInitDone := TRUE;
END_IF;

```

B.5.1.2 Inputs

In the 'Inputs' section the hardware and function block input signals are processed, this includes filtering and scaling.

```

(*****
Inputs
*****)
(* Input filtering thermal protection *)
s_fbDtThermal(
    I_bInput := iq_stIO.I_bThMotorOK,
    i_stConfig := i_stConfig.stThermalProtection,
    iq_stHmi := iq_stHmi.stThermalProtection,);

(* Input filtering running feedback *)
s_fbFbRunning(
    I_bInput := iq_stIO.I_FbRunning,
    iq_stConfig := i_stConfig.stRunningFeedback,
    iq_stHmi := iq_stHmi.stRunningFeedback,);

```

B.5.1.3

Alarm

In the 'alarm' section the error conditions of the CM are monitored. Also, the alarms are collected and mapped to an internal error state.

```

(*****
Alarms
*****)
(* Thermal protection alarm *)
s_fbAlmThermalProtection(
    i_bTrigger := i_stControl.bEnable AND s_fbDtThermal.q_bStaActive,
    i_bReset := i_stControl.bReset,
    i_stConfig := i_stConfig.stAlmThermalProtection,
    q_stStaActive := q_stAlarms.bThermalProtection);

(* Running feedback alarms *)
s_fbAlmRunningFeedback(
    i_bTrigger := i_stControl.bEnable
    AND s_fbRunMotor.Q_bOutput AND NOT(s_fbFbRunning.q_bStaActive),
    i_bReset := i_stControl.bReset,
    i_stConfig := i_stConfig.stAlmRunningFeedback,
    q_stStaActive := q_stAlarms.bRunningFeedback);

(* Collect Alarms *)
s_bStaError := q_stAlarms.bThermalProtection OR q_stAlarms.bRunningFeedback;

```

B.5.1.4

HMI control

The HMI has only 2 buttons: a start and stop command. When the manual control is not allowed the buttons are disabled and the manual request is reset.

```

(*****
  Hmi Control
  *****)
(* Allow manual control *)
s_bAlwManualControl := i_stControl.bReqEnable
                      AND i_stControl.bAlwManualControl AND NOT(s_bStaError);

(* Hmi button stop motor *)
s_fbHmiBtnStopMotor(
  i_bEnable := s_bAlwManualControl,
  i_bFeedback := FALSE,
  iq_stHmi := iq_stHmi.stBtnStopMotor, );

(* Hmi button run motor *)
s_fbHmiBtnRunMotor(
  i_bEnable := s_bAlwManualControl,
  i_bFeedback := s_bCmdRunManual,
  iq_stHmi := iq_stHmi.stBtnRunMotor, );

(* Set internal command to run in manual operation *)
IF NOT(s_bAlwManualControl) OR s_fbHmiBtnStop.q_bStaActiveOsr THEN
  s_bCmdRunManual := FALSE;

ELSIF s_fbHmiBtnRunMotor.q_bStaActiveOsr THEN
  s_bCmdRunManual := TRUE;
END_IF

```

B.5.1.5**Logic**

In the logic section the actual state of the CM is determined. Based on the external requests the motor is activated if the 'run' and 'enable' are high and the 'stop' is low, and if there is no internal error active.

```

(*****
  Logic
  *****)
IF NOT(i_stControl.bReqEnable) OR s_bStaError THEN
  s_bCmdRun := FALSE;
  s_bCmdRunManual := FALSE;

ELSIF i_stControl.bReqStop THEN
  s_bCmdRun := FALSE;
  s_bCmdRunManual := FALSE;

ELSIF i_stControl.bReqRun THEN
  s_bCmdRun := TRUE;
END_IF

```

B.5.1.6**Outputs**

The motor has only 1 physical output, a running contact. In this section also the CM's output signals are determined.

```
(*****
Outputs
*****)
(* Hardware outputs *)
s_fbRunMotor(
    i_bReqStatus := (s_bCmdRun OR s_bCmdRunManual),
    i_stConfig := i_stConfig.stRunMotor,
    Q_bOutput => iq_stIO.Q_bRunMotor,);

(* Function block outputs *)
q_bStaEnabled := i_stControl.bReqEnabled;
q_bStaRunning := (s_bCmdRun OR s_bCmdRunManual) AND s_fbFbRunning.q_bStaActive;
q_bStaStandStill := s_tonStandStill.Q;
```

B.5.1.7**Statistics**

In this example there is only one statistical value, the running hours of the motor.

```
(*****
Statistics
*****)
(* Running hours *)
s_fbRunningHours(
    i_bCount := q_bStaRunning,
    i_bReset := s_bResetStatistics );
```

B.5.1.8**HMI status**

In the section 'HMI status' all relevant data and status information is processed and copied to an HMI friendly structure. In this example we have 2 LED's and a statistic.

```
(*****
Hmi Status
*****)
(* Hmi LED motor running *)
s_fbHmiLedMotorRunning(
    i_bStatus := q_bStaRunning,
    iq_stHmi := iq_stHmi.stLedMotorRunning );

(* Hmi LED motor standstill *)
s_fbHmiLedMotorStandStill(
    i_bStatus := q_bStaStandStill,
    iq_stHmi := iq_stHmi.stLedMotorStandStill );

(* Statistic Running hours *)
iq_stHmi.fStatRunningHours := s_fbRunningHours.q_fValue;
```